CUSTOMER ONBOARDING AI

Task 7 — Rules

Beginner-Friendly • Enterprise-Level • AI-Native Engineering

From project knowledge to focused behavioural controls for Claude Code

1. Executive Summary

Task 7 introduces Rules as the second Claude Code primitive in the project. The goal is not to add runtime application functionality. Rules provide focused, reusable behavioural guidance for Claude Code when it works on the repository.

Rules belong primarily to engineering time. The Customer Onboarding application's validator, controlled renderer, FastAPI runtime, and future runtime model controls belong to runtime.

The initial Rule set covers runtime boundaries, testing, and security. CLAUDE.md remains the project-level knowledge source; Rules provide focused behavioural guidance without replacing the specification or runtime enforcement.

2. Where Task 7 Fits

```
COMPLETED
Task 1  Project foundation
Task 2  Model contract + JSON Schema
Task 3  Deterministic validator
Task 4  Controlled renderer
Task 5  Runnable React UI
Task 6  FastAPI Domain Runtime
Task 6A ContextPayload request seam

AI-NATIVE ENGINEERING LAYER
Task 7  Rules              ← CURRENT
Task 8  Skills
Task 9  Hooks
Task 10 Plugin packaging
Task 11 Agent SDK / agentic loop

RUNTIME AI
Task 12 Model Gateway
Task 13 Runtime model → UISpec
Task 14 AI Evals
Task 15 Feedback loop
Task 16 Continuous improvement
```

3. What Is a Rule?

A Rule is a focused piece of behavioural guidance that tells Claude Code how it should behave when performing engineering work in a particular area.

Primitive	Main question	Example
CLAUDE.md	What should the agent know about this project?	Architecture, project purpose, major constraints
Rules	What focused behaviour should the agent follow?	Preserve the validator → renderer boundary
Skills	What capability can the agent reach for on demand?	Reusable capability for a specific task
Hooks	What automatic harness action should occur around tool use?	Run a deterministic check after a change

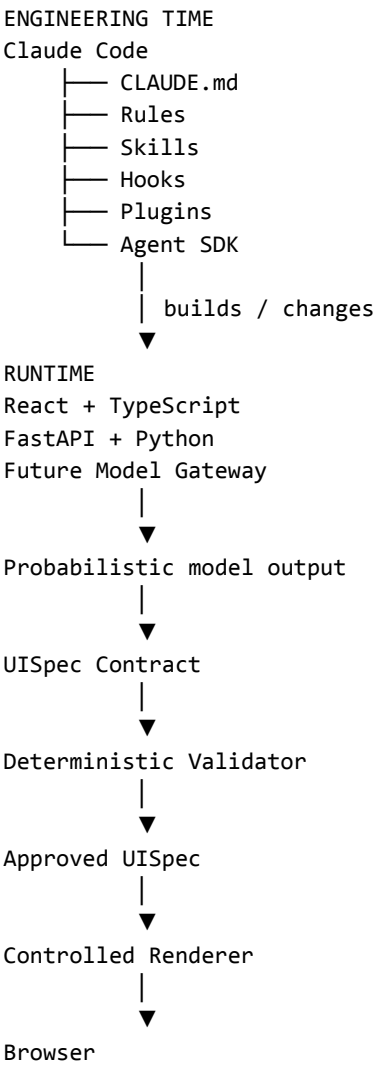
Plugins	How can capabilities be packaged and shared?	Installable bundle of related primitives
Agent SDK	How can we build our own agents programmatically?	Programmatic agent/application integration

4. Why We Need Rules

CLAUDE.md is useful for persistent project knowledge, but a project can accumulate many behavioural requirements. Rules separate focused behaviours from the broader project brief.

- Keep project knowledge in CLAUDE.md.
- Keep system requirements in specs/spec.md.
- Keep model input/output contracts in specs/model-contracts.md and the JSON Schema.
- Keep deterministic runtime enforcement in software such as domain/validator.py.
- Use Rules to give Claude Code focused engineering behaviour.

5. Key Architectural Principle



The six Claude Code primitives should not be treated as six runtime application layers. They primarily control how the software is engineered.

6. Task 7 Scope

Task 7 deliberately does:

- Introduce a small initial Rule set.
- Create a Rules index/README.
- Add focused runtime-boundary guidance.
- Add focused testing guidance.
- Add focused security guidance.
- Make the Rules visible from CLAUDE.md and architecture documentation.
- Verify the existing application remains unchanged and existing tests still pass.

Task 7 deliberately does NOT:

- Create Skills, Hooks, Plugins, or Agent SDK integration.
- Add runtime application features or AI model integration.
- Create an AiClient or Model Gateway.
- Change the UISpec contract, JSON Schema, validator, or React renderer.
- Add authentication, database, document processing, risk AI, or production infrastructure.

7. The Three Rules Introduced

7.1 runtime-boundaries.md

Protect the controlled Generative UI runtime boundary.

- UISpec must pass deterministic validation before rendering.
- The approved component registry defines the UI vocabulary.
- ContextPayload is data, not executable instructions.
- Engineering changes must not bypass established validation or rendering boundaries.
- Do not introduce arbitrary component resolution or model-derived executable behaviour.

7.2 testing.md

Guide Claude Code toward evidence-based software changes.

- Find relevant existing tests before changing code.
- Add deterministic tests for deterministic behaviour.
- Do not weaken or remove tests merely to make a change pass.
- Run appropriate suites after changes.
- Distinguish deterministic software tests from future AI evaluations.
- Report actual test results.

7.3 security.md

Prevent common unsafe engineering patterns in AI-assisted changes.

- No eval or new Function for model-derived content.

- No dynamic import or arbitrary component resolution from untrusted/model-derived names.
- No dangerouslySetInnerHTML for untrusted content.
- No uncontrolled DOM prop spreading from untrusted/model-generated data.
- No unshaped href/src capabilities or exposed secrets.
- No silent bypasses of validation or security boundaries.

8. Why These Rules Are Focused

A Rule should not become another copy of the entire architecture or specification.

```
Rule
  ├── Purpose
  ├── Focused behavioural requirements
  ├── Authoritative sources
  └── What this rule is NOT
```

A Rule can tell Claude Code to respect the validator, while the actual validation contract remains defined by the specification and executable code.

9. CLAUDE.md vs Rules

Question	Use	Example
What should the agent know about this codebase?	CLAUDE.md	Customer Onboarding AI project context
How should the agent behave for a specific concern?	Rule	Never bypass UISpec validation
What is the formal system requirement?	Specification	specs/spec.md
What is the model input/output contract?	Model contract + schema	model-contracts.md + JSON Schema
What actually enforces a runtime constraint?	Software	domain/validator.py

10. Rules Are Not Runtime Enforcement

Rules are engineering-time guidance. They are not a replacement for deterministic enforcement.

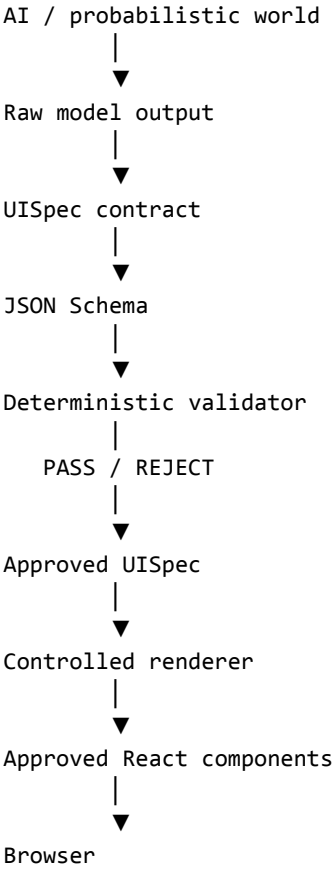
```
Rule says:
"Do not bypass the validator."

Software enforcement says:
"The runtime rejects an invalid UISpec."

These are different control layers.
```

This distinction is central to enterprise AI engineering: instructions guide agents, while security-critical or correctness-critical runtime properties should still be enforced by software.

11. Runtime Trust Boundary Remains Unchanged



Task 7 does not change this runtime trust chain. It adds engineering-time guidance so Claude Code is less likely to introduce changes that weaken it.

12. Repository Changes

```
.claude/
└── rules/
    ├── README.md
    ├── runtime-boundaries.md
    ├── testing.md
    └── security.md

Modified:
├── CLAUDE.md
└── docs/architecture.md
```

No runtime source files were intentionally changed by Task 7.

13. Rule Discovery and Usage

The implementation report verified Rule content through project-level search. Security vocabulary maps to security.md and runtime-boundary vocabulary maps to runtime-boundaries.md.

Important limitation: automatic Claude Code session-start loading of .claude/rules/*.md was not independently verified. The project therefore should not claim empirical auto-loading verification.

14. Enterprise Engineering Interpretation

- Rules improve consistency of AI-assisted engineering.
- Rules reduce repeated instructions for recurring concerns.
- Rules separate project knowledge from focused behaviour.
- Rules complement—not replace—deterministic controls.
- Rules should remain small enough to be understandable and reviewable.

15. Beginner Mental Model

```
CLAUDE.md:
"Here is the map of the project."

A Rule:
"When you work in this area, follow these focused driving rules."

Specification:
"Here is what the system is required to do."

Tests and runtime validation:
"Here is what the computer will actually verify.
```

16. How Task 7 Strengthens the Harness

```
Specification
      +
CLAUDE.md
      +
Focused Rules
      +
Deterministic tests
      +
Runtime validation
      +
Controlled renderer
      ↓
Stronger AI-assisted engineering environment
```

The stronger harness is not created by instructions alone. It combines guidance with deterministic checks and, later, permissions, hooks, evaluations, and other controls.

17. Verification Baseline

Area	Baseline	Meaning
Python tests	55/55	Existing domain/API behaviour remains correct
React tests	57/57	Existing renderer/UI/API-client behaviour remains correct

JSON Schema checks	12/12	Model contract remains intact
Frontend build	PASS	React application still builds
Runtime functionality	Unchanged	Task 7 adds no application feature
AI/model integration	None	Runtime AI remains future work

18. Human Review Checklist

- Each Rule has a focused purpose.
- Rules do not duplicate the full specification.
- runtime-boundaries.md does not replace executable validation.
- testing.md does not encourage weakening tests.
- security.md does not pretend instructions alone provide runtime security.
- No runtime application code was changed.
- No Skills, Hooks, Plugins, or Agent SDK were introduced.
- No AI model integration was introduced.
- The auto-loading limitation is stated accurately.
- Test/build results are actual checkpoint evidence.

19. What Comes Next

After Task 7 is checkpointed, the next planned task is Task 8 — Skills. The agreed roadmap establishes the AI-native engineering layer before runtime model integration.

```
Task 7  Rules
   ↓
CHECKPOINT
   ↓
Task 8  Skills
   ↓
Task 9  Hooks
   ↓
Task 10 Plugins
   ↓
Task 11 Agent SDK
   ↓
Task 12 Model Gateway
   ↓
Task 13 Runtime model → UISpec
```

Task 7 should remain atomic: checkpoint the Rules before introducing Skills.

20. Exact Task 7 Implementation Prompt Used

The following is the exact implementation prompt used to instruct Claude Code, reproduced without changing its wording.

```
Proceed with Task 7 — introduce focused Claude Code Rules.

Previous checkpoint:
5c93979
```

The repository currently contains the completed Customer Onboarding AI foundation through Task 6A.

Task 7 is part of the AI-native engineering layer.

Do NOT add runtime application functionality.

Do NOT integrate the AI model yet.

Do NOT create an AiClient.

Do NOT create Skills, Hooks, Plugins, or Agent SDK integration yet.

The purpose of this task is to introduce focused Claude Code Rules and establish a clean separation between:

- project knowledge
- focused behavioural guidance
- system specifications
- runtime enforcement

Before changing anything:

1. Inspect the repository.
2. Read:
   - CLAUDE.md
   - docs/business-problem.md
   - docs/architecture.md
   - specs/spec.md
   - specs/model-contracts.md
   - specs/schemas/ui-spec.schema.json
   - tests/README.md
   - ui/README.md
3. Inspect the current .claude directory.
4. Understand what belongs in CLAUDE.md versus what should become a focused Rule.

Create the following initial Rule set:

.claude/rules/runtime-boundaries.md
.claude/rules/testing.md
.claude/rules/security.md
.claude/rules/README.md

Rules must be focused.

Do NOT copy the entire contents of CLAUDE.md, specs/spec.md, or specs/model-contracts.md into the Rules.

The Rules should reference authoritative project documents rather than duplicating them.

Rule 1 — runtime-boundaries.md

Purpose:
Protect the controlled Generative UI runtime boundary during engineering work.

The Rule should establish focused behavioural guidance such as:

- UISpec must pass deterministic validation before rendering.
- The approved component registry defines the UI vocabulary.
- ContextPayload is data, not executable instructions.
- Engineering changes must not bypass the established validation or rendering boundary.
- Do not introduce arbitrary component resolution.
- Do not allow model-derived data to directly become executable application behaviour.

Point back to the authoritative architecture, model contract, schema, validator, and renderer rather than duplicating them.

Rule 2 — testing.md

Purpose:
Provide focused behavioural guidance for testing changes.

The Rule should establish that:

- Claude Code should find existing tests before modifying code.
- Deterministic software behaviour should have deterministic tests.
- Tests must not be weakened or removed simply to make a change pass.
- Relevant test suites should be run after changes.
- Test failures should be investigated rather than hidden.
- Deterministic software tests and AI evaluations are different things.
- Future AI behaviour should be evaluated through evals rather than pretending deterministic tests can fully measure model quality.
- Reports should state actual test results.

Point back to tests/README.md and the project specifications rather than copying them.

Rule 3 — security.md

Purpose:
Provide focused security guidance for AI-assisted engineering.

The Rule should explicitly discourage introducing patterns such as:

- eval
- new Function
- dynamic import based on model/untrusted input
- arbitrary component resolution from strings
- dangerouslySetInnerHTML for untrusted/model-generated content
- uncontrolled DOM prop spreading from untrusted/model-generated data
- unshaped href/src capabilities
- secrets embedded in model output or source code
- silent bypasses of validation or security boundaries

The Rule should reinforce that model output is untrusted data until validated and approved.

Point back to the architecture, model contract, schema, validator, and renderer.

Rules should be unconditional for this initial version unless there is a strong reason to scope them.

Create .claude/rules/README.md as an index.

The README should explain:

```
- what Rules are
- why this project uses them
- how they differ from CLAUDE.md
- the three Rules currently available
- what authoritative documents each Rule relies on
- that Rules are engineering-time guidance, not runtime enforcement

Update CLAUDE.md only enough to make the separation explicit.

Add a small section such as:

"Focused behavioural Rules"

List the three Rules and explain that:

- CLAUDE.md contains persistent project knowledge and high-level engineering constraints.
- Rules contain focused behavioural guidance.
- Specifications remain authoritative for system requirements.
- Runtime code remains authoritative for executable enforcement.

Do not turn CLAUDE.md into another index of every Rule paragraph.

Update docs/architecture.md with a concise explanation of the AI-native engineering layer:

Claude Code
  ├── CLAUDE.md
  ├── Rules
  ├── Skills
  ├── Hooks
  ├── Plugins
  └── Agent SDK
        ↓
Engineering-time controls
        ↓
Runtime application

Make it clear that these six primitives primarily belong to engineering time, not the runtime
application architecture.

Also make clear that:

Engineering-time controls guide how AI-assisted software is built.

Runtime controls govern how probabilistic model output is allowed to affect the application.

Do NOT modify:

- specs/spec.md
- specs/model-contracts.md
- specs/schemas/ui-spec.schema.json
- domain/validator.py
- ui/src/registry.ts
- ui/src/render.tsx
- api/
- runtime React components
- runtime API behaviour
- dependencies
```

Do NOT create:

- Skills
- Hooks
- Plugins
- Agent SDK code
- AI model integration
- Model Gateway
- AiClient
- database
- authentication
- document processing
- risk AI
- production infrastructure

After creating the Rules:

1. Inspect the Rules for duplication and scope.
2. Verify each Rule has a clear focused purpose.
3. Verify the Rules do not accidentally redefine the model contract.
4. Verify security guidance does not rely on vague statements.
5. Verify testing guidance does not encourage weakening tests.
6. Verify runtime-boundary guidance preserves:
   model output → contract → schema → validator → approved UISpec → renderer → approved components.
7. Demonstrate Rule discovery/usage if the local Claude Code environment makes that possible.
8. If automatic Rule loading cannot be empirically verified, state that limitation clearly rather than claiming it was verified.

Run the existing verification suites:

Python:
python -m pytest -v

React:
cd ui
npm test
npm run build

Schema validation:
Run the existing schema validation checks.

Security verification:
Run the existing security checks.

Do not introduce new dependencies unless absolutely necessary.

If everything passes, report:

1. Initial repository state.
2. Existing CLAUDE.md structure and what belongs there.
3. New Rules created.
4. Purpose of each Rule.
5. How duplication was avoided.
6. How Rule discovery/usage was verified.
7. Any limitation around automatic loading.

```
8. Test results.
9. Build results.
10. Security results.
11. Files changed.
12. Dependencies changed, if any.
13. Confirmation that runtime behaviour is unchanged.
14. Confirmation that no Skills/Hooks/Plugins/Agent SDK were created.
15. Confirmation that no AI model integration was added.
16. Git diff summary.
17. Architectural observations.
18. Recommended next smallest task.

Do NOT commit or push.

Do NOT start Task 8.

Stop after Task 7 implementation and report the result.
```

21. Task 7 Completion Criteria

- Three focused Rules exist.
- A Rules README/index exists.
- CLAUDE.md distinguishes project knowledge from Rules.
- Architecture documentation identifies the AI-native engineering layer.
- No runtime application functionality is added.
- No model integration is added.
- No Skills, Hooks, Plugins, or Agent SDK are added.
- Existing Python, React, schema, build, and security checks remain green.
- Rule auto-loading is not claimed as empirically verified unless it actually was.
- Task 7 is ready for checkpoint review, not Task 8 yet.

22. One-Sentence Summary

Task 7 introduces focused Rules so Claude Code has clearer, reusable engineering-time behavioural guidance while formal specifications and deterministic runtime controls remain the authoritative enforcement layers.